

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	DECLARATION	i
	ACKNOWLEDGEMENT	ii
	LIST OF FIGURES	iii
	ABSTRACT	iv
1.	INTRODUCTION	1-4

2.	PROBLEM IDENTIFICATION AND	5-11
	FEASIBILITY STUDY	
	2.1. Problem Identification	5
	2.2 Feasibility Study	6
	2.2.1. Technical Feasibility	7
	2.2.2. Economic Feasibility	8
3.	2.2.3. Operational Feasibility	9
	2.2.4. Legal and Ethical Feasibility	9
	2.2.5. Market Feasibility	10
	REQUIREMENT ANALYSIS	12-20
	3.1. Functional Requirements	13
	3.1.1. User Authentication and Registration	13
	3.1.2. Inventory Management	13
	3.1.3. Sales and Purchase Management	14
	3.1.4. Reporting and Analytics	14
	3.1.5. Notifications and Alerts	14
	3.1.6. User Profile Management	15
	3.2. Non-Functional Requirements	15
	3.3. System Requirements	16
	3.3.1. Hardware Requirements	17

3.3.2. Software Requirements	17
3.4. Data Requirements	17

3.5. Technology used	Error! Bookmark not defined.
----------------------	-------------------------------------

3.5.1. React.js	Error! Bookmark not defined.
-----------------	-------------------------------------

18

3.5.2. Tailwind CSS

18

3.5.3. HTML & CSS

19

3.5.4. Node.js & Express.js	19
3.5.5. MongoDB	19
3.5.6. JWT (Json Web Token)	19
3.5.7. VS Code (Virtual Studio Code)	
4. REVIEW OF PREVIOUS WORK	21-31
4.1. Madamidola et al. (2024)	25
4.2. Puppala Sridhar et al. (2021)	28
5. PROJECT DESCRIPTION	
5.1. Project Overview	32
5.2. Objective & Scope	32
5.2.1. Objective	33
5.2.2. Scope	34
5.3. Technologies Used	34
5.3.1. Frontend	34
5.3.2. Backend	34
5.3.3. Database	35
5.3.4. Tools & DevOps	35
5.4. Project Overflow	35
5.4.1. User Authentication	35
5.4.2. Inventory Management	35

	5.4.3. Order and Sales Tracking	36
	5.4.4. Reporting and Analytics	36
	5.4.5. Notifications and Alerts	36
	5.5. Detailed Features	36
	5.5.1. Real-Time Inventory Tracking	36
	5.5.2. Sales & Purchase Management	37
	5.5.3. Admin Panel	37
	5.5.4. Responsive UI	37
	5.5.5. Reports & Dashboards	37
6.	DESIGN	38-53
		38
	6.1. Data Flow Diagram	
		40
	6.2. Activity Diagram	
		41
	6.3. Class Diagram	
		42
	6.4. Snapshots of the project	
7.	CONCLUSION AND FUTURE WORK	54-61
	REFERENCES	62
	TEAM DETAILS	63-64

LIST OF FIGURES

FIGURE NUMBER	FIGURE NAME	PAGE NUMBER
3.1	Gantt Chart	12
6.1	Data Flow Diagram (DFD)	39
6.2	Activity Diagram	40
6.3	Class Diagram	41
6.4	Homepage	45
6.5	Features of the website: User	45
Authentication and E-Commerce		
6.6	Features of the website:	46
Dashboard and Cart		
6.7	Features of the website: Alerts	46
6.8	Signup Page	47
6.9	Login Page	47
6.10	Profile Page: Admin	48
6.11	Admin Dashboard	48
6.12	Admin's Add Product Feature	49
6.13	Admin's Update Product Feature	49
6.14	Admin's Product History Page	50

6.15	Profile Page: User	50
6.16	Products Section	51
6.17	Purchasing product through cart	51
6.18	User Cart	52
6.19	Order History	52
6.20	Attributions of resources	53

Abstract

In today's digital world, document fraud and tampering have become serious issues in areas such as education, government records, and business transactions. Traditional document verification systems are often centralized, making them vulnerable to data manipulation, unauthorized access, and security breaches. To overcome these challenges, this project presents a **Blockchain Based Document Verification Web Application** that ensures authenticity, integrity, and security of digital documents.

The proposed system uses blockchain technology to store the hash value of documents, which acts as a unique digital fingerprint. When a document is uploaded, its hash is generated using

cryptographic algorithms and stored securely on the blockchain. During verification, the system compares the hash of the uploaded document with the stored hash. If both hashes match, the document is verified as original; otherwise, it is identified as tampered.

This web application is developed using modern full-stack technologies, where the frontend provides a user-friendly interface for uploading and verifying documents, and the backend handles data processing and interaction with the blockchain. A lightweight database such as SQLite is used to manage document-related metadata efficiently.

The key advantages of this system include enhanced security, transparency, immutability, and trust. Since blockchain data cannot be altered once recorded, it ensures that documents remain tamper-proof. This system can be widely used in universities for certificate verification, in organizations for document validation, and in government sectors for secure record management.

In conclusion, the Blockchain Based Document Verification Web Application provides a reliable and efficient solution to prevent document fraud and ensures the authenticity of digital records in a decentralized and secure manner.

CHAPTER 1

INTRODUCTION

In the modern digital era, the use of electronic documents has increased rapidly across various sectors such as education, government organizations, healthcare, banking, and corporate industries. Certificates, identity proofs, contracts, and official records are now commonly stored and shared in digital form. While this transformation has improved efficiency and accessibility, it has also introduced serious challenges related to **document security, authenticity, and trust**. One of the major problems faced today is **document forgery and tampering**, where fake or altered documents are presented as genuine, leading to fraud, financial loss, and loss of credibility.

Traditional document verification systems are mostly **centralized**, meaning that all data is stored and managed by a single authority or server. These systems suffer from several limitations. Firstly, they are vulnerable to cyberattacks and unauthorized access, which can result in data manipulation or leakage. Secondly, centralized systems often lack transparency, making it difficult for users to independently verify the authenticity of documents. Thirdly, the verification process is usually time-consuming and may require manual intervention, which reduces efficiency and increases operational cost.

To address these challenges, the concept of **blockchain technology** has emerged as a powerful and innovative solution. Blockchain is a **decentralized and distributed ledger system** that records data in a secure, transparent, and immutable manner. Once data is stored on the blockchain, it cannot be altered or deleted, ensuring a high level of data integrity. Each record (block) is linked to the previous one using cryptographic techniques, forming a chain that is resistant to tampering.

This project, titled “**Blockchain Based Document Verification Web Application,**” aims to leverage the features of blockchain to create a secure and reliable system for verifying digital documents. The core idea behind this system is to generate a **cryptographic hash** of each document, which acts as a unique digital signature. Even a small change in the document results in a completely different hash value, making it easy to detect any form of tampering.

When a user uploads a document, the system generates its hash using a secure hashing algorithm and stores it on the blockchain. The actual document may be stored off-chain (e.g., in a database like SQLite), while the hash ensures its authenticity. During the verification process, the user uploads the document again, and the system recalculates its hash and compares it with the stored hash on the blockchain. If both values match, the document is considered original; otherwise, it is flagged as tampered or invalid.

The web application is designed using a **full-stack development approach**, where the frontend provides an intuitive and user-friendly interface for uploading and verifying documents, and the backend handles business logic, database operations, and blockchain interaction. Technologies such as **Node.js** are used for server-side development, while **SQLite** is used as a lightweight database to manage document metadata efficiently.

The proposed system offers several advantages over traditional methods. It ensures **data security** through cryptographic hashing, **transparency** through decentralized verification, and **immutability** through blockchain storage. Additionally, it reduces dependency on manual verification processes, thereby saving time and resources. The system can be effectively used in various real-world applications such as **educational certificate verification, legal document validation, identity authentication, and secure record management in government and private sectors.**

In conclusion, this project provides a modern, secure, and efficient solution to the growing problem of document fraud. By integrating blockchain technology with web development, it enhances trust, reliability, and transparency in digital document verification systems, making it highly relevant in today's data-driven world.

CHAPTER 2

PROBLEM IDENTIFICATION AND FEASIBILITY STUDY

2.1. Problem Identification

In the current digital environment, the rapid shift from physical to electronic documents has created significant challenges in ensuring the **authenticity, security, and reliability** of documents. Various sectors such as education, government, banking, and corporate industries depend heavily on digital documents like certificates, identity proofs, contracts, and official records. However, with the increasing use of digital platforms, the risk of **document forgery, duplication, and tampering** has also grown substantially. Identifying and addressing these issues is the primary motivation behind this project.

One of the major problems is the **ease of document manipulation**. Digital documents can be easily edited using modern software tools without leaving visible traces. This makes it difficult for organizations to determine whether a document is original or has been altered. For example, fake

educational certificates or modified legal documents can be used for fraudulent purposes, leading to serious consequences such as job fraud, financial loss, and legal disputes.

Another critical issue is the **lack of a reliable verification system**. Traditional verification methods often involve manual processes such as contacting issuing authorities, checking physical records, or relying on third-party verification agencies. These methods are not only time-consuming but also inefficient and prone to human error. In many cases, verification can take days or even weeks, which delays important decision-making processes.

The **centralized nature of existing systems** is also a major concern. Most current document management and verification systems store data on a single server or controlled database. This creates a **single point of failure**, meaning that if the server is hacked, corrupted, or goes offline, all stored data can be compromised or lost. Additionally, centralized systems are more vulnerable to cyberattacks, unauthorized access, and data breaches, which can lead to manipulation or theft of sensitive information.

Another problem is the **lack of transparency and trust**. Users often have to rely entirely on the authority managing the system, without having the ability to independently verify the authenticity of documents. This creates trust issues, especially in scenarios involving multiple parties such as employers, universities, and government agencies. There is no easy way for a third party to verify whether a document is genuine without depending on intermediaries.

Furthermore, there is an issue of **data integrity**. In traditional systems, records can be modified or deleted by administrators or attackers without proper traceability. This makes it difficult to track changes or detect unauthorized modifications. As a result, maintaining the integrity of documents over time becomes a significant challenge.

The **high cost and inefficiency** of current verification processes also add to the problem. Organizations often spend considerable resources on verification procedures, including manpower, time, and infrastructure. This reduces overall productivity and increases operational expenses.

In summary, the key problems identified are:

- Easy tampering and forgery of digital documents
- Lack of fast and reliable verification methods
- Security risks due to centralized systems
- Absence of transparency and trust among users
- Poor data integrity and traceability
- High cost and time consumption in verification processes

These challenges highlight the need for a more **secure, transparent, efficient, and tamper-proof system**, which leads to the adoption of blockchain technology in this project for document verification.

2.2. Feasibility Study

A feasibility study is an important step in system development that evaluates whether a proposed project is **practical, achievable, and beneficial**. It helps in analyzing different aspects such as technical requirements, cost, operational efficiency, and time constraints before actual implementation. For the project “**Blockchain Based Document Verification Web Application,**” the feasibility study ensures that the system can be successfully developed and deployed in a real-world environment.

2.2.1 Technical Feasibility

Technical feasibility examines whether the required technology, tools, and skills are available to build the system.

In this project, the system is developed using modern and widely used technologies such as **Node.js** for backend development, along with HTML, CSS, and JavaScript for the frontend. A lightweight database like **SQLite** is used to store document-related data efficiently. For implementing blockchain functionality, libraries and frameworks (such as Web3 or custom blockchain logic) can be integrated.

All these technologies are open-source, well-documented, and supported by large developer communities. This makes development easier and reduces technical risks. Additionally, the system does not require highly specialized hardware, as it can run on standard computers and servers.

Therefore, from a technical perspective, the project is **highly feasible**.

2.2.2 Economic feasibility

Economic feasibility focuses on the **cost-benefit analysis** of the project.

The proposed system is cost-effective because:

- It uses open-source technologies (no licensing cost)
- Requires minimal hardware infrastructure
- Reduces manual verification costs

- Saves time and operational expenses in the long run

Although there may be some initial development and deployment costs, the long-term benefits such as **reduced fraud, improved efficiency, and automation** outweigh these expenses. Organizations can save money by eliminating third-party verification services and reducing manpower.

Hence, the project is **economically feasible**.

2.2.3. Operational Feasibility

Operational feasibility determines whether the system will function effectively in a real-world environment and whether users will accept it.

The proposed web application is designed with a **simple and user-friendly interface**, allowing users to easily upload and verify documents without requiring technical expertise. The process is automated, reducing manual effort and making it faster and more reliable.

Organizations such as universities, companies, and government offices can easily adopt this system because it improves transparency and trust. Minimal training is required for users to operate the system.

Thus, the system is **operationally feasible** and practical for real-world use.

2.2.4. Legal Feasibility

Legal feasibility checks whether the project complies with laws and regulations.

This system ensures that:

- User data is handled securely
- No unauthorized access or misuse of documents occurs
- Digital verification processes follow standard data protection practices

Blockchain technology enhances data security and integrity, which aligns with modern data protection requirements. However, organizations must ensure compliance with local data protection laws when deploying the system.

Overall, the project is **legally feasible**, provided proper data handling policies are followed.

2.2.5. Schedule (Time) Feasibility

Schedule feasibility evaluates whether the project can be completed within the given time frame.

This project can be developed in phases, such as:

- Designing frontend interface
- Developing backend logic
- Implementing blockchain integration
- Testing and deployment

Since the technologies used are simple and well-known, the development process can be completed within a reasonable time (especially for a college project). Proper planning and task management ensure timely completion.

Therefore, the project is **time feasible**.

Conclusion

The feasibility study clearly indicates that the **Blockchain Based Document Verification Web Application** is practical and achievable. It is technically sound, cost-effective, easy to operate, legally compliant, and can be completed within the given time. Hence, the project is **feasible and suitable for real-world implementation**, offering a secure and efficient solution to document verification problems.

CHAPTER 3

REQUIREMENT ANALYSIS

Requirement Analysis

Requirement Analysis is a crucial phase in system development where all the **needs, expectations, and functionalities** of the proposed system are identified and clearly defined. It helps in understanding what the system should do, how it should perform, and what resources are required to build it. For the project **“Blockchain Based Document Verification Web Application,”** requirement analysis ensures that the system is designed according to user needs and works efficiently in real-world scenarios.

1. Functional Requirements

Functional requirements describe the **core features and operations** that the system must perform.

a) User Registration and Login

The system should allow users (such as admin or authorized personnel) to securely register and log in. Authentication ensures that only authorized users can upload or verify documents.

b) Document Upload

Users should be able to upload documents (such as PDFs or images) through the web interface. The system should accept valid file formats and store the document securely.

c) Hash Generation

Once a document is uploaded, the system must generate a **cryptographic hash value**. This hash acts as a unique digital fingerprint of the document.

d) Blockchain Storage

The generated hash value should be stored on the blockchain. This ensures that the data is **immutable (cannot be changed)** and tamper-proof.

e) Document Verification

The system should allow users to upload a document for verification. It will generate a new hash and compare it with the stored hash on the blockchain.

- If both hashes match → Document is **original**
- If hashes do not match → Document is **tampered**

f) Database Management

A database (like SQLite) should store document-related metadata such as file name, upload date, user details, and hash reference.

g) Result Display

The system should clearly display the verification result (Original / Tampered) to the user in a simple and understandable format.

2. Non-Functional Requirements

Non-functional requirements define the **quality and performance aspects** of the system.

a) Security

- Data must be protected using encryption and secure protocols
- Only authorized users should access the system
- Blockchain ensures data integrity and prevents tampering

b) Performance

- The system should process document uploads and verification quickly
- It should handle multiple users efficiently without delay

c) Scalability

- The system should be able to handle increasing number of users and documents in the future

d) Usability

- The interface should be simple, clean, and user-friendly
- Users should not require technical knowledge to use the system

e) Reliability

- The system should provide accurate verification results every time
- It should work consistently without failure

f) Maintainability

- The system should be easy to update and maintain
 - Code should be structured and modular
-

3. Hardware Requirements

The system does not require high-end hardware. Basic requirements include:

- Computer/Laptop with internet connection
 - Minimum 4GB RAM
 - Storage for documents and database
-

4. Software Requirements

The software components required for development include:

- **Node.js** for backend development
 - HTML, CSS, JavaScript for frontend
 - **SQLite** for database management
 - Blockchain libraries (such as Web3 or custom implementation)
 - Web browser (Chrome, Edge, etc.)
-

5. User Requirements

The system should meet the needs of different users:

- **Admin/User:** Upload and manage documents
 - **Verifier:** Check document authenticity
 - **Organizations:** Ensure secure and trusted document validation
-

Conclusion

Requirement analysis defines the complete structure and working of the **Blockchain Based Document Verification Web Application**. By clearly identifying functional and non-functional requirements, along with hardware and software needs, it ensures that the system is **secure, efficient, user-friendly, and reliable**. This step acts as a foundation for successful system design and implementation

CHAPTER 4

REVIEW OF PREVIOUS WORK

The concept of document verification has evolved significantly over time, moving from traditional manual methods to modern digital and blockchain-based solutions. Understanding previous work in this area is essential to identify existing limitations and justify the need for a more secure and efficient system like the **Blockchain Based Document Verification Web Application**.

1. Traditional (Manual) Document Verification Systems

In earlier times, document verification was carried out manually by authorities. For example, educational institutions verified certificates by contacting universities, and organizations checked documents through physical records or official letters.

Limitations:

- **Time-consuming process:** Verification could take days or even weeks
- **Human errors:** Manual checking often led to mistakes
- **Lack of scalability:** Difficult to handle large volumes of documents
- **No real-time verification:** Immediate validation was not possible

These issues made traditional systems inefficient and unreliable in today's fast-paced digital world.

2. Digital Document Verification Systems

With the advancement of technology, many organizations shifted to digital platforms. Documents are now stored in databases and verified using online systems.

Working:

- Documents are uploaded and stored in centralized databases
- Verification is done by matching records in the database

Limitations:

- **Centralized storage** creates a single point of failure

- Vulnerable to **cyberattacks and data breaches**
- Data can be **modified or deleted** by administrators or hackers
- Limited transparency for users

Although digital systems improved speed and accessibility, they still lack strong security and trust mechanisms.

3. Use of Cryptographic Techniques

Some systems introduced cryptographic methods such as **hashing and digital signatures** to enhance security.

Working:

- A document is converted into a hash value (unique digital fingerprint)
- Any small change in the document changes the hash completely

Advantages:

- Improved detection of document tampering
- Better data integrity

Limitations:

- Hash values are usually stored in centralized databases
- Still dependent on trusted third parties
- No guarantee against data manipulation in storage systems

4. Blockchain-Based Solutions (Recent Developments)

Recent research and projects have started using **blockchain technology** for document verification. Blockchain provides a decentralized and immutable system for storing data.

Working:

- Document hash is stored on blockchain
- Each transaction is recorded permanently
- Verification is done by comparing hash values

Advantages:

- **Decentralization** eliminates single point of failure
- **Immutability** ensures data cannot be changed
- **Transparency** allows independent verification
- **High security** using cryptographic principles

Limitations in Existing Blockchain Systems:

- Some systems are complex and difficult to use
 - High implementation cost in certain blockchain networks
 - Limited user-friendly interfaces
 - Scalability issues in large-scale deployments
-

5. Research Papers and Existing Projects

Various researchers and developers have proposed systems for secure document verification:

- Some projects focus on **educational certificate verification** using blockchain
- Others use blockchain for **identity management and digital records**
- Many systems integrate blockchain with web applications for better accessibility

However, most of these systems:

- Are **theoretical or prototype-based**
 - Lack full implementation in real-world scenarios
 - Do not focus on **simple user experience**
 - Require technical knowledge to operate
-

6. Gap Analysis (What is Missing in Previous Work)

After analyzing previous systems, several gaps are identified:

- Lack of **fully secure and tamper-proof system**
 - Over-dependence on **centralized databases**
 - Absence of **simple and user-friendly interfaces**
 - Limited **real-time verification capability**
 - High cost and complexity in some blockchain solutions
-

7. Need for Proposed System

Considering the limitations of previous work, there is a clear need for a system that:

- Uses **blockchain for secure and immutable storage**
- Provides **fast and real-time document verification**
- Is **cost-effective and easy to use**
- Ensures **data integrity, transparency, and trust**

The proposed **Blockchain Based Document Verification Web Application** addresses these gaps by combining blockchain technology with a simple web interface and efficient backend processing.

Conclusion

The review of previous work shows that while significant progress has been made in document verification systems, existing solutions still face challenges related to **security, trust, transparency, and usability**. Traditional and digital systems are not fully reliable, and even current blockchain-based solutions have certain limitations. Therefore, the proposed system aims to overcome these issues by providing a **secure, efficient, and user-friendly platform** for document verification, making it a valuable improvement over previous approaches.

CHAPTER 5

PROJECT DESCRIPTION

Title: Blockchain Based Document Verification Web Application

The **Blockchain Based Document Verification Web Application** is a secure, reliable, and user-friendly system designed to verify the authenticity of digital documents and prevent forgery or tampering. In today's digital world, where documents such as certificates, contracts, identity proofs, and official records are frequently shared online, ensuring their originality has become a major challenge. This project provides a modern solution by combining **web technologies with blockchain principles** to create a tamper-proof and transparent verification system.

1. Overview of the System

The main objective of this project is to build a web-based platform where users can **upload, store, and verify documents securely**. The system uses a **cryptographic hashing mechanism** along with blockchain technology to ensure that documents cannot be altered once they are registered.

Instead of storing the entire document on the blockchain (which can be costly and inefficient), the system stores only the **hash value of the document**. This hash acts as a unique digital fingerprint. Even a minor change in the document will produce a completely different hash, making tampering easy to detect.

2. Working Principle of the System

The system operates in two main phases:

a) Document Registration (Upload Phase)

When a user uploads a document:

- The system accepts the document (PDF, image, etc.)
- A **hash value** is generated using a cryptographic algorithm
- This hash is stored on the blockchain
- The document itself is stored in a database like **SQLite**
- Metadata such as file name, upload date, and user details are also stored

This ensures that the original document's identity is securely recorded.

b) Document Verification Phase

When a document needs to be verified:

- The user uploads the document again
- The system generates a new hash value
- It compares this hash with the hash stored on the blockchain

Result:

- If both hashes match → Document is **Original**
- If hashes do not match → Document is **Tampered**

This process is fast, automated, and highly reliable.

3. System Architecture

The project follows a **three-tier architecture**:

a) Frontend (User Interface)

- Built using HTML, CSS, and JavaScript
 - Provides a simple and interactive interface
 - Allows users to upload and verify documents easily
 - Displays verification results clearly
-

b) Backend (Server Side)

- Developed using **Node.js**
 - Handles business logic, file processing, and hash generation
 - Communicates with the database and blockchain layer
 - Ensures secure handling of user requests
-

c) Blockchain Layer

- Stores hash values in a decentralized manner
 - Ensures immutability and transparency
 - Prevents unauthorized modification of data
-

d) Database Layer

- Uses **SQLite**
 - Stores document metadata and file references
 - Provides efficient data retrieval
-

4. Key Features of the System

a) Tamper Detection

Any change in the document can be instantly detected through hash comparison.

b) Security

- Uses cryptographic hashing
- Blockchain ensures data cannot be altered

c) Transparency

Users can verify documents independently without relying on third parties.

d) Fast Verification

Verification is completed within seconds, unlike traditional methods.

e) User-Friendly Interface

Designed for easy use without requiring technical knowledge.

5. Modules of the System

1. User Module

- Registration and login
- Upload and verify documents

2. Document Management Module

- Handles document storage and retrieval
- Maintains metadata

3. Hash Generation Module

- Generates unique hash values for documents

4. Blockchain Module

- Stores and retrieves hash values securely

5. Verification Module

- Compares hash values and displays results

6. Advantages of the System

- Eliminates document fraud and forgery
- Provides **tamper-proof verification**
- Reduces manual effort and time

- Enhances trust between users and organizations
 - Cost-effective and scalable solution
-

7. Applications of the System

This system can be used in multiple domains:

- **Education Sector:** Verification of certificates and mark sheets
 - **Government Sector:** Secure record management
 - **Corporate Sector:** Employee document verification
 - **Banking & Finance:** KYC and identity verification
 - **Legal Sector:** Contract validation
-

8. Limitations of the System

- Requires internet connectivity
 - Initial setup of blockchain may require technical knowledge
 - Scalability challenges in large-scale blockchain networks
-

9. Future Enhancements

- Integration with advanced blockchain platforms (like Ethereum)
- Adding **smart contracts** for automated verification

- Multi-user role management (admin, verifier, organization)
 - Cloud storage integration for better scalability
 - Mobile application development
-

Conclusion

The **Blockchain Based Document Verification Web Application** is a powerful solution to the growing problem of digital document fraud. By combining **blockchain technology, cryptographic hashing, and web development**, the system ensures that documents remain **secure, authentic, and tamper-proof**. It improves efficiency, reduces verification time, and builds trust among users and organizations. This project demonstrates how modern technologies can be effectively used to solve real-world problems in a simple and practical way.

CHAPTER 6

DESIGN

The **Design** of the **VerifyBlock** system is centered on a decentralized architecture that replaces traditional centralized databases with an immutable Ethereum blockchain ledger to ensure absolute document integrity. The core logic follows a linear flow where a document's unique digital identity is established using the **SHA-256 hashing algorithm**, creating a 64-character hexadecimal "fingerprint" that changes entirely if even a single pixel or character in the file is altered. Architecturally, the system

is divided into a user-friendly frontend for file interaction, a processing layer for real-time hash generation, and a blockchain backend for permanent storage and verification of these hashes. By comparing the live hash of an uploaded file against the historical record stored on the blockchain, the design provides an automated, transparent, and tamper-proof method for authenticating digital assets

6.1. Context Diagram

A **Context Diagram** is the highest-level diagram of a system that shows the **overall structure and interaction** between the system and external entities. It represents the entire system as a **single process (one block)** and focuses only on **inputs and outputs**, not internal details.

For your project “**Blockchain Based Document Verification Web Application,**” the context diagram helps to understand **how users and external systems interact** with your application.

1. Main Components of Context Diagram

a) Central System (Main Process)

At the center of the diagram is the main system:

Blockchain Based Document Verification Web Application

This represents your complete website which handles:

- Document upload
- Hash generation
- Blockchain storage

- Document verification

b) External Entities

These are the actors that interact with your system:

1. Users (Registered Users)

- Upload documents
- Request document verification
- Receive verification results

2. Administrator

- Manages system
- Can verify documents
- Monitors data and users

3. Document Database

- Stores uploaded documents
- Stores metadata (file name, date, etc.)

4. Blockchain Network

- Stores document hash values
 - Ensures immutability and security
-

2. Data Flow (Input & Output)

Context diagram mainly shows **data flow** between entities and system:

Input to System:

- Document Upload (from user)
- Verification Request (from user/admin)

Output from System:

- Verification Result (Original / Tampered)
- Confirmation of document storage

System Interaction with Storage:

- Stores document in database
- Stores hash in blockchain
- Retrieves hash during verification

3. Working Flow (Simple Understanding)

1. User uploads document → System receives it
2. System generates hash
3. Hash stored in blockchain
4. Document stored in database
5. User requests verification
6. System compares hashes

7. Result sent back to user

4. Key Features of Context Diagram

- Shows **big picture only** (no internal steps)
- Represents system as **single process**
- Highlights **external interactions**
- Easy to understand and explain in exams

5. Why Context Diagram is Important

- Helps in **understanding system boundaries**
- Identifies **external users and systems**
- Useful for **system design and planning**
- Makes project explanation **clear and simple**



Figure 6.1. Context Diagram

6.2. Data Flow Diagram

A **Data Flow Diagram (DFD)** is a graphical representation that shows **how data moves inside a system**. It explains how data is **input, processed, stored, and output** in a structured way. Unlike the context diagram (which shows only the overall view), DFD provides **detailed internal working** of the system.

For your project “**Blockchain Based Document Verification Web Application,**” the DFD explains how documents are uploaded, processed, stored, and verified step by step.

1. Main Components of DFD

a) External Entities

These are outside the system but interact with it:

- **User** → uploads and verifies documents
 - **Administrator** → manages and verifies documents
-

b) Processes

Processes are the **main operations** performed by the system:

1. **Upload Document (Process 1.0 / 3.0)**
 - Accepts document from user
 2. **Generate Hash (Process 2.0 / 4.0)**
 - Creates a unique hash value for the document
 3. **Save Document (Process 3.0 / 5.0)**
 - Stores document and metadata
 4. **Verify Document (Process 4.0 / 6.0)**
 - Compares hash values and generates result
-

c) Data Stores

These are storage locations where data is saved:

- **Document** **Database**
→ Stores files and metadata
 - **Blockchain** **Network**
→ Stores hash values securely
-

d) Data Flow

Arrows represent movement of data:

- Document data
 - Hash values
 - Verification results
-

2. Working of DFD (Step-by-Step)

Step 1: Document Upload

- User uploads document
 - Data flows to **Upload Process**
-

Step 2: Hash Generation

- Document is sent to **Generate Hash Process**
 - System creates a unique hash
-

Step 3: Save Data

- Document stored in **Database**
 - Hash stored in **Blockchain**
-

Step 4: Verification Request

- User/Admin requests verification
 - Document is uploaded again
-

Step 5: Hash Comparison

- New hash is generated
 - Retrieved hash from blockchain
 - Both hashes are compared
-

Step 6: Result Output

- If match → **Original**
 - If not match → **Tampered**
 - Result sent back to user
-

3. Levels of DFD

DFD Level 0 (Basic)

- Shows main processes only
- Simple overview

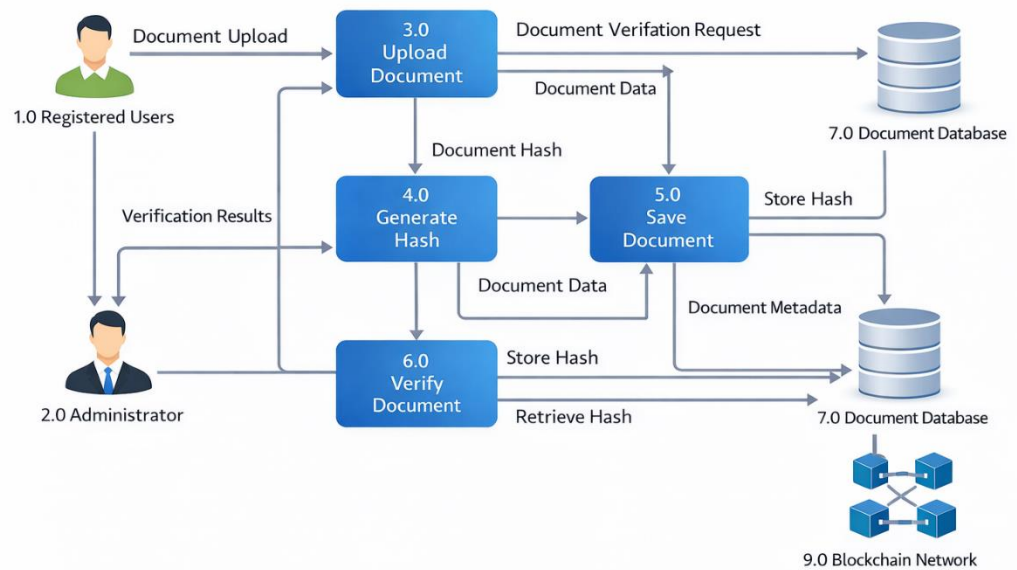
DFD Level 1 (Detailed)

- Shows all internal processes
- Used in your diagram

4. Key Features of DFD

- Shows **complete data movement**
- Explains system **step-by-step**
- Easy to understand visually
- Helps in system design

Data Flow Diagram – Blockchain Based Document Verification Web Application



6.2. Data Flow Diagram

6.3. Entity Relationship Diagram

An **Entity Relationship Diagram (ERD)** is used to represent the **structure of a database**. It shows how different **entities (tables)** are related to each other along with their **attributes (fields)** and **relationships**.

For your project “**Blockchain Based Document Verification Web Application,**” the ER diagram explains how data like users, documents, blockchain records, and verification results are connected.

1. Main Components of ER Diagram

a) Entities (Tables)

Entities are the main objects in the system (represented as rectangles):

1. Users

Stores	user	details
--------	------	---------

Attributes:

- user_id (Primary Key)
- username
- email
- password
- registration_date

2. Documents

Stores	uploaded	document	details
--------	----------	----------	---------

Attributes:

- document_id (Primary Key)
- user_id (Foreign Key)
- document_name
- document_hash
- upload_date

3. Verification Records

Stores verification history

Attributes:

- verification_id (Primary Key)
- document_id (Foreign Key)
- verification_date
- verification_status

4. Blockchain

Stores hash values securely

Attributes:

- block_id (Primary Key)
- document_hash

2. Relationships in ER Diagram

Relationships define how entities are connected:

1. User → Documents (One-to-Many)

☞ One user can upload multiple documents

- Relationship: **“uploads”**
 - Example: One student uploads many certificates
-

2. Documents → Blockchain (One-to-One)

☞ Each document has one hash stored in blockchain

- Relationship: **“stores hash”**
 - Ensures document security
-

3. Documents → Verification Records (One-to-Many)

☞ One document can be verified multiple times

- Relationship: **“verifies”**
 - Each verification stored separately
-

3. Keys in ER Diagram

Primary Key (PK)

- Unique identifier for each record
- Example: `user_id`, `document_id`

Foreign Key (FK)

- Connects one table to another
 - Example: `user_id` in Documents
-

4. Working of ER Diagram (Simple Flow)

1. User registers and logs in
 2. User uploads document → stored in Documents table
 3. Hash is generated → stored in Blockchain table
 4. When verification happens → record stored in Verification table
 5. System uses relationships to fetch and compare data
-

5. Why ER Diagram is Important

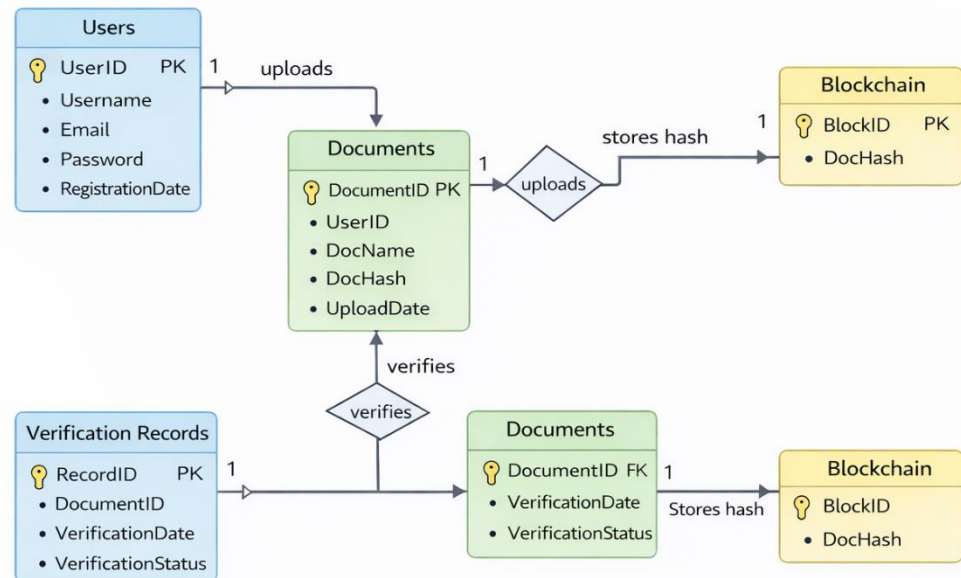
- Helps in **database design**
 - Shows **data relationships clearly**
 - Avoids data redundancy
 - Makes development easier
 - Important for **viva and exams**
-

6. Key Features of Your ER Diagram

- Clear separation of entities
- Proper use of primary and foreign keys

- Supports blockchain-based security
- Maintains verification history

Entity Relationship Diagram –
Blockchain Based Document Verification Web Application



6.3. Entity Relationship Diagrams

6.4 Flowchart

A **flowchart** is a diagram that represents the **step-by-step working process of a system** using standard symbols like **oval (start/end)**, **rectangle (process)**, **diamond (decision)**, and **arrows (flow)**. It helps in understanding how a system works logically from beginning to end.

For your project “**Blockchain Based Document Verification Web Application,**” the flowchart shows how a document is uploaded, processed, and verified.

1. Purpose of Flowchart

- To explain the **logic of the system clearly**
- To show **step-by-step execution**
- To make system easy to understand for developers and examiners

2. Symbols Used in Flowchart

- **Oval** → Start / End
- **Rectangle** → Process (operation)
- **Diamond** → Decision (Yes/No condition)
- **Arrow** → Flow of steps

3. Step-by-Step Working of Flowchart

Step 1: Start

- The process begins when the user opens the web application

Step 2: Upload Document

- User uploads a document (PDF/image)
 - System receives the file
-

Step 3: Generate Hash

- System generates a **unique hash value** of the document
 - This acts as a digital fingerprint
-

Step 4: Store Data

- Document is stored in database
 - Hash is stored in blockchain
-

Step 5: Verification Request

- User wants to verify a document
 - Uploads the document again
-

Step 6: Generate New Hash

- System generates a new hash for the uploaded document
-

Step 7: Compare Hashes (Decision Step)

- System compares:
 - New hash (current document)
 - Stored hash (blockchain)

☞ **Decision (Diamond):**

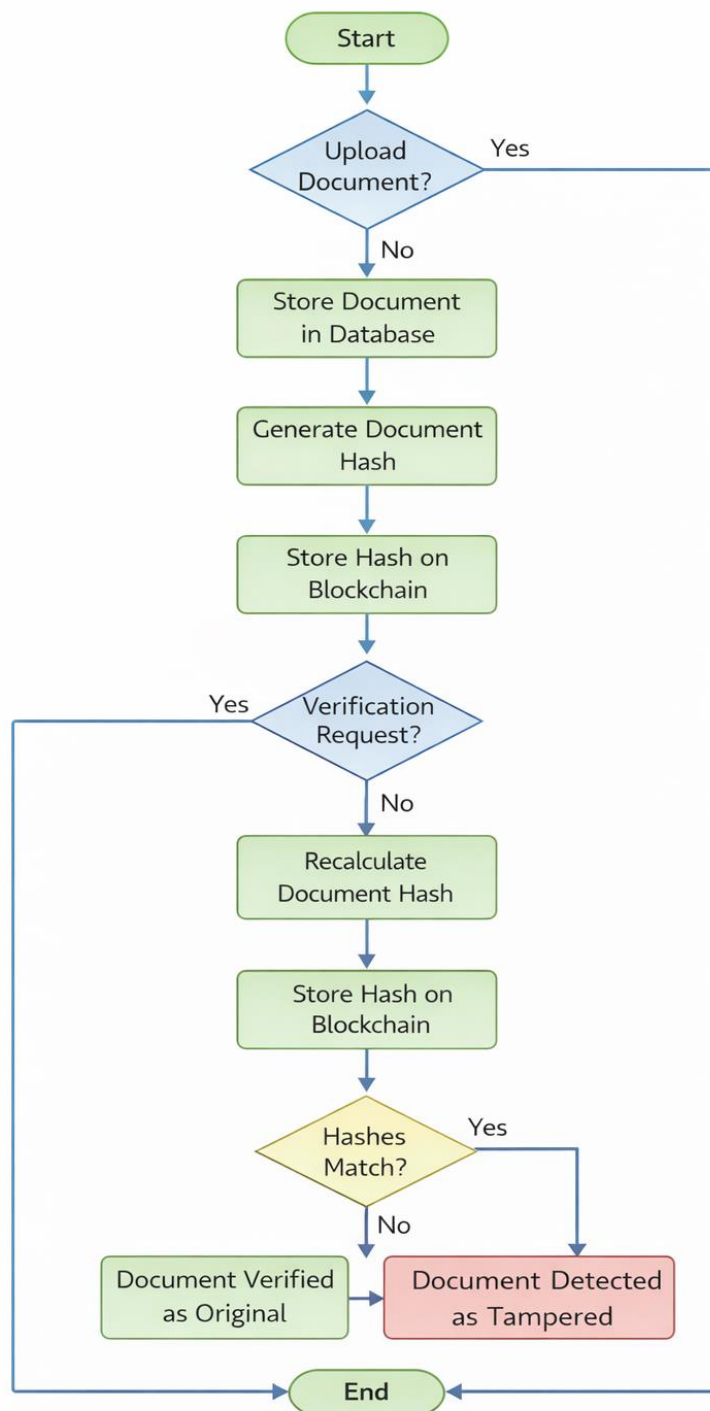
- If both hashes match → YES
- If not → NO

Step 8: Display Result

- ✓ If YES → Document is **Original**
- ✗ If NO → Document is **Tampered**

Step 9: End

- Process ends after displaying result



6.4. Flowchart Diagram

6.5. Snapshots of the Project

The **VerifyBlock** application is a decentralized web platform designed to ensure document integrity. It utilizes the **SHA-256** hashing algorithm and the **Ethereum blockchain** to create a tamper-proof verification system.

6.5.1 Home Page (User Interface)

The Home Page serves as the primary entry point for the user.

- **Visual Design:** It features a clean, professional blue-and-white theme with a clear navigation bar containing links to Home, Upload, Verify, and Dashboard.
- **Functionality:** The central "Get Started" button directs users to the core document processing features.

6.5.2 Upload Document Module

This module is the first step in the security process, where a document is transformed into a digital fingerprint.

- **Working Process:** The user selects a file from their local system.
- **Hashing Engine:** Upon selection, the system automatically computes a unique **SHA-256 Hash**.
- **Output:** A success message is displayed along with the generated hash string (e.g., b1e7de1afc0a...), which is then recorded on the blockchain for permanent storage.

6.5.3 Verify Document Module

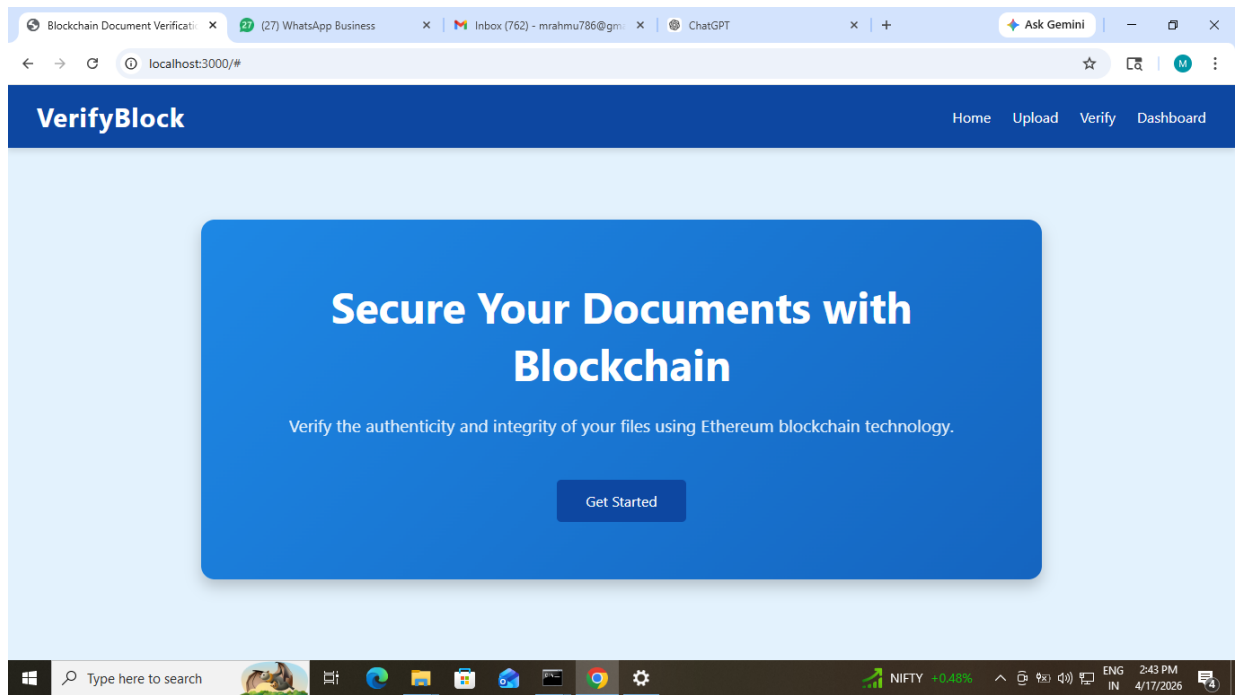
This is the core functional component that detects whether a file is authentic or has been altered.

- **Step-by-Step Logic:** The user uploads a document to be checked and clicks "Check Authenticity".
- **Verification Status:**
 - **Original Document:** If the live hash of the file matches the hash stored on the blockchain, the system displays a green success box with a checkmark.
 - **Tampered Document:** If even a single character in the document has been changed, the hashes will not match, and the system displays a red warning box with a cross icon.

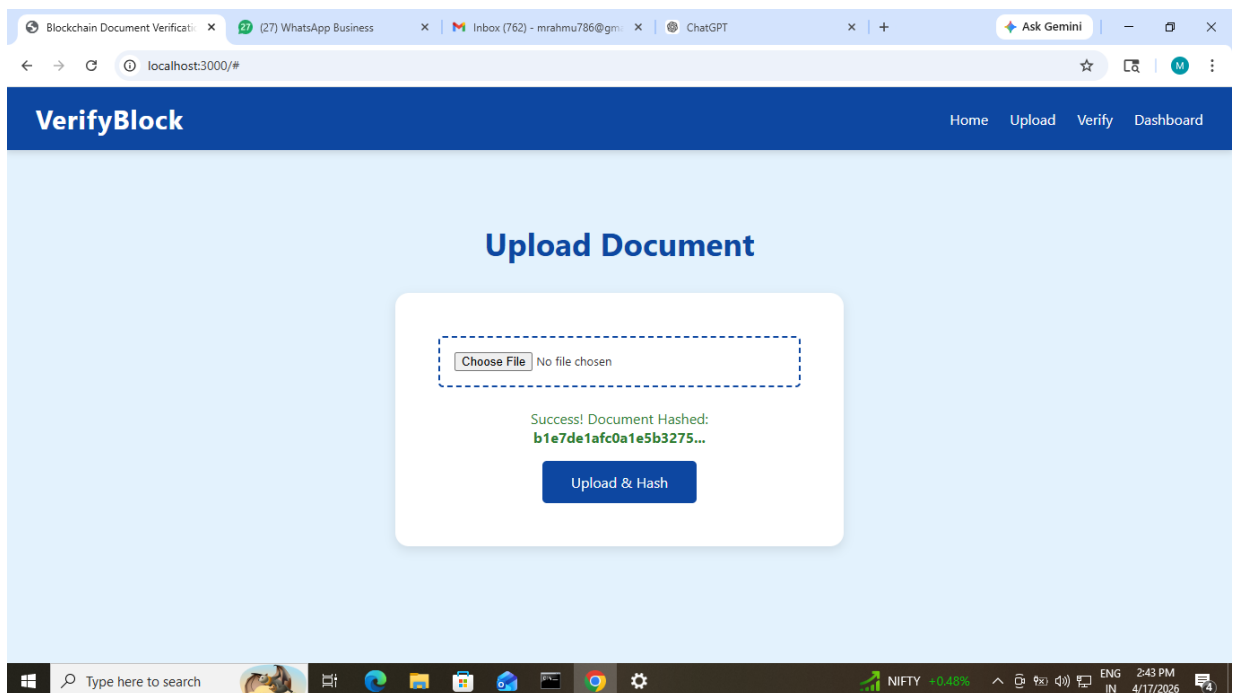
6.5.4 Document History (The Dashboard)

The Dashboard provides a transparent ledger of all activities recorded on the system.

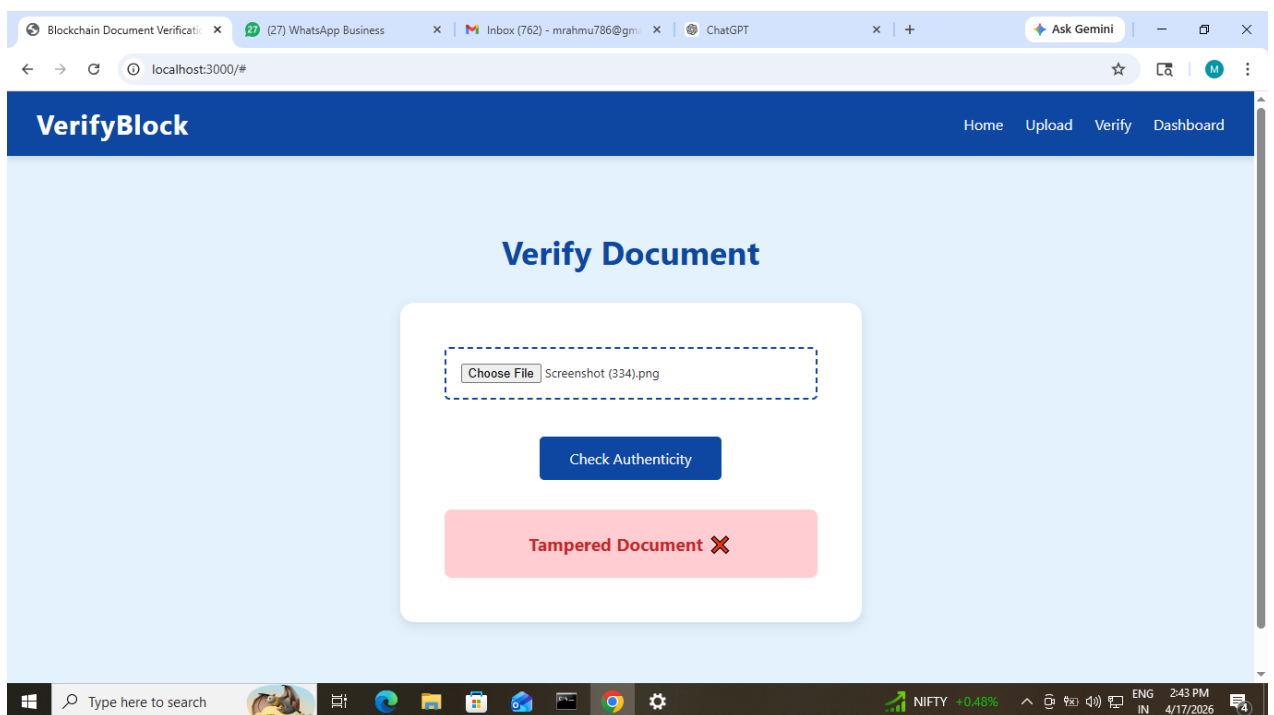
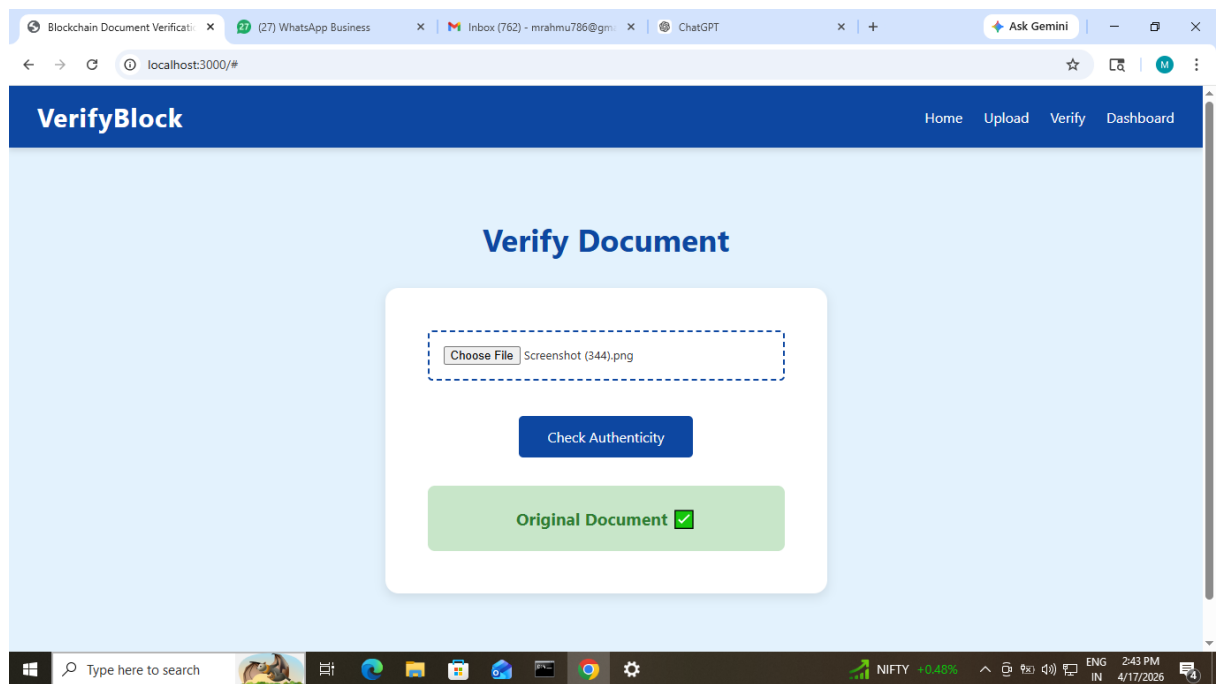
- **Data Presentation:** Information is presented in a structured table containing the **File Name**, **Hash (SHA-256)**, and the precise **Upload Date**.
- **Audit Trail:** This module allows users and administrators to verify when a specific document (such as zoyacv.pdf) was secured and what its original fingerprint was.



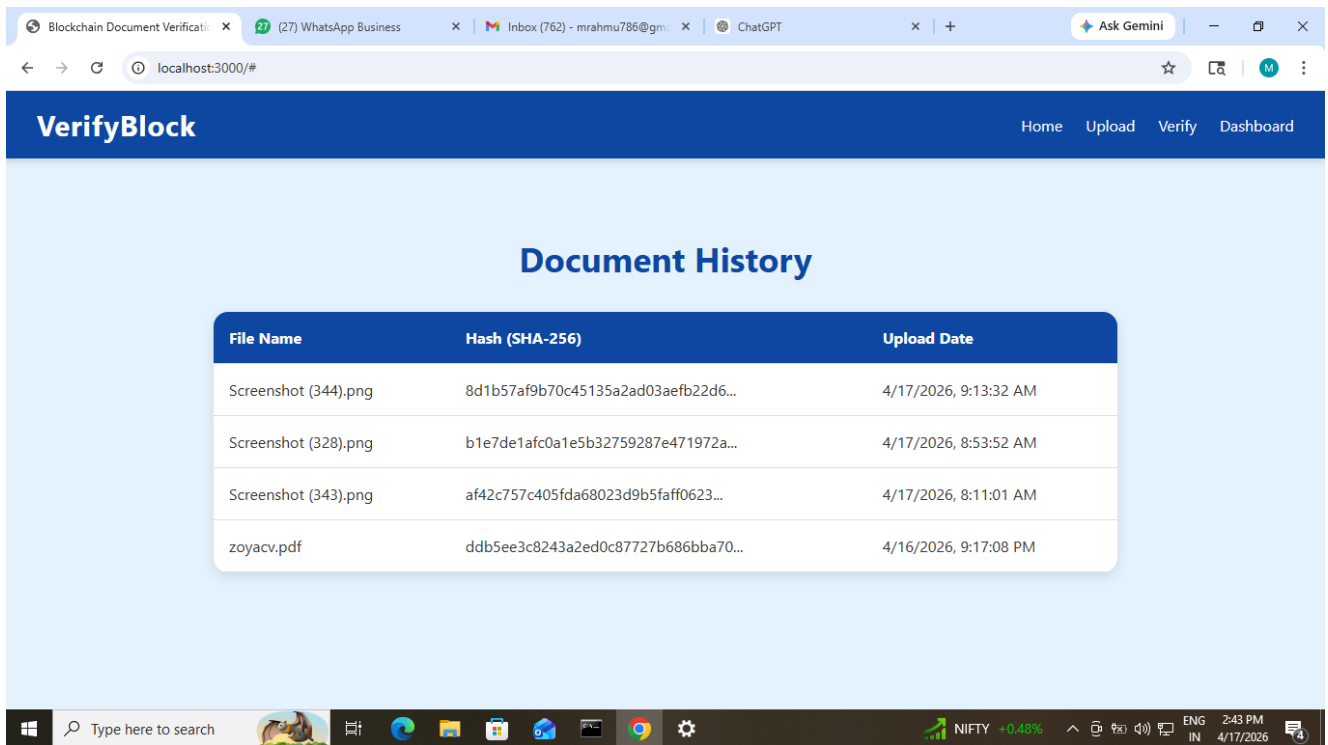
6.5.1. Home Page



6.5.2. Upload Document Module



6.5.3. Verify Document Module



6.5.4. Document History (Dashboard)

6.6. Database & Tables

In your project “**Blockchain Based Document Verification Web Application,**” the database plays a very important role. It is used to **store, manage, and retrieve data** related to users, documents, blockchain hashes, and verification results.

A **database** is an organized collection of data, and **tables** are used to store this data in structured form (rows and columns). You are using **SQLite**, which is simple, fast, and perfect for a college project.

1. Database Overview

Database Name:

document_verification.db

This database contains multiple tables that are connected with each other using relationships.

2. Tables in the Database

1. Users Table

Purpose:

Stores all user and admin information.

Fields:

- user_id → Unique ID (Primary Key)
- username → User name
- email → Unique email
- password → User password
- role → user/admin
- created_at → Registration date

Example: One user can upload multiple documents.

2. Documents Table**Purpose:**

Stores uploaded document details.

Fields:

- document_id → Primary Key
- user_id → Foreign Key (linked to Users)
- file_name → Name of document
- file_path → Storage location
- document_hash → Unique hash
- upload_date → Upload time

Important: This is the main table for document tracking.

3. Blockchain Records Table**Purpose:**

Stores hash values in blockchain-like structure.

Fields:

- block_id → Primary Key
- document_id → Foreign Key
- hash_value → Document hash
- previous_hash → Previous block hash
- timestamp → Time of storage

Important: Ensures **data immutability**.

4. Verification Records Table**Purpose:**

Stores verification history.

Fields:

- verification_id → Primary Key

- document_id → Foreign Key
- user_id → Foreign Key
- uploaded_hash → New hash
- stored_hash → Original hash
- status → Original / Tampered
- verification_date → Date

Important: Keeps track of all verification attempts.

3. Relationships Between Tables

- **Users** → **Documents** → One-to-Many
 - **Documents** → **Blockchain** → One-to-One
 - **Documents** → **Verification Records** → One-to-Many
 - This structure ensures proper data connection and avoids duplication.
-

4. SQL Code (Create Tables)

```
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    role TEXT DEFAULT 'user',
```

```
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    );
```

```
CREATE TABLE Documents (
    document_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    file_name TEXT NOT NULL,
    file_path TEXT NOT NULL,
    document_hash TEXT NOT NULL,
    upload_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

```
CREATE TABLE Blockchain_Records (
    block_id INTEGER PRIMARY KEY AUTOINCREMENT,
    document_id INTEGER,
    hash_value TEXT NOT NULL,
    previous_hash TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (document_id) REFERENCES Documents(document_id)
);
```

```
CREATE TABLE Verification_Records (
    verification_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
document_id INTEGER,  
user_id INTEGER,  
uploaded_hash TEXT,  
stored_hash TEXT,  
status TEXT,  
verification_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (document_id) REFERENCES Documents(document_id),  
FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

5. Conclusion

The database design of your project:

- Ensures **secure storage of data**
 - Maintains **relationships between entities**
 - Supports **blockchain-based verification**
 - Provides **fast and accurate results**
-

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1. Conclusion

The **Blockchain Based Document Verification Web Application** successfully addresses one of the most critical challenges in the digital era—ensuring the **authenticity and integrity of documents**. With the increasing use of digital documents across various sectors such as education, government, and corporate industries, the risk of document forgery and tampering has also grown significantly. Traditional verification systems, which are mostly centralized and manual, are not only time-consuming but also vulnerable to security breaches and data manipulation.

This project provides a modern and efficient solution by integrating **blockchain technology with web-based systems**. The use of cryptographic hashing ensures that each document has a unique identity, and storing this hash on a blockchain makes the system **immutable and tamper-proof**. Even the slightest modification in the document can be detected instantly through hash comparison, which enhances the reliability of the verification process.

The system is designed using a full-stack approach, where the frontend offers a simple and user-friendly interface, and the backend handles all processing tasks such as file handling, hash generation, and database management. The use of **Node.js** and **SQLite** makes the system lightweight, efficient, and easy to deploy. Additionally, the separation of document storage and hash storage improves performance and security.

Overall, this project demonstrates how emerging technologies like blockchain can be effectively used to build **secure, transparent, and trustworthy systems**. It reduces dependency on third-party verification, saves time, minimizes human effort, and increases confidence among users and organizations. The system is scalable and can be adapted to various real-world applications such as certificate verification, legal documentation, and identity management.

7.2. Future Work

Although the current system provides a strong foundation for secure document verification, there are several enhancements that can be implemented in the future to make it more advanced, scalable, and widely applicable.

One major improvement can be the integration of **public blockchain platforms such as Ethereum**, which would make the system more decentralized and globally accessible. By using smart contracts, the verification process can be further automated, reducing the need for manual intervention and increasing efficiency. Smart contracts can also enable automatic approval or rejection of documents based on predefined conditions.

Another area of enhancement is **scalability and performance optimization**. As the number of users and documents increases, the system can be migrated from SQLite to more robust databases like MySQL or cloud-based solutions. Additionally, integrating **cloud storage services** can improve file storage, accessibility, and backup capabilities.

The system can also be extended by adding **multi-user roles and access control**, such as admin, verifier, organization, and public users. This will allow different levels of permissions and improve system management. Implementing advanced authentication mechanisms like **two-factor authentication (2FA)** will further enhance security.

Another important future enhancement is the development of a **mobile application**, which will make the system more accessible and user-friendly. Users will be able to upload and verify documents directly from their smartphones, increasing usability and adoption.

The integration of **Artificial Intelligence (AI)** can also be explored to detect fraudulent documents based on patterns and anomalies, adding an extra layer of security beyond hash comparison. Furthermore, features like **QR code-based verification** can be implemented, where each document is assigned a QR code that can be scanned for instant verification.

Finally, the system can be expanded to support **interoperability with other systems and institutions**, enabling seamless verification across organizations and countries. This would make the platform more powerful and suitable for large-scale deployment.

In conclusion, while the current project provides a strong and secure solution for document verification, future enhancements can transform it into a **comprehensive, scalable, and globally accepted verification system**, leveraging the full potential of blockchain and modern technologies.

9.1. References

Mastering Blockchain

→ Used for understanding blockchain concepts, working, and applications.

Blockchain Basics

→ Helped in learning fundamental blockchain principles in simple terms.

Node.js Official Documentation

→ Used for backend development and server-side logic.

SQLite Documentation

→ Used for database design and implementation.

MDN Web Docs

→ Used for HTML, CSS, and JavaScript concepts.

Ethereum Documentation

→ Used for understanding blockchain and smart contracts.

Research Papers on:

- Blockchain in document verification
- Digital signature and hashing techniques

Online Tutorials & Resources:

- YouTube tutorials on full stack development
- GitHub repositories for blockchain projects

9.2. Appendices

APPENDIX A: SOURCE CODE

This section contains the important parts of the source code used in the development of the **Blockchain Based Document Verification Web Application**. The application is developed using **Node.js** for backend and HTML, CSS, and JavaScript for frontend.

A.1 Frontend Code (HTML)

<!DOCTYPE html>

<html>

<head>

<title>Document Verification</title>

</head>

<body>

<h2>Upload Document</h2>

<form action="/upload" method="POST" enctype="multipart/form-data">

<input type="file" name="document" required>

<button type="submit">Upload</button>

</form>

<h2>Verify Document</h2>

<form action="/verify" method="POST" enctype="multipart/form-data">

<input type="file" name="document" required>

<button type="submit">Verify</button>

</form>

</body>

</html>

A.2 Backend Code (Node.js)

const express = require('express');

const multer = require('multer');

const crypto = require('crypto');

const sqlite3 = require('sqlite3').verbose();

const app = express();

const upload = multer({ dest: 'uploads/' });

const db = new sqlite3.Database('document_verification.db');

// Hash function

function generateHash(filePath) {

const fs = require('fs');

const fileBuffer = fs.readFileSync(filePath);

```

    return crypto.createHash('sha256').update(fileBuffer).digest('hex');
  }

// Upload API

app.post('/upload', upload.single('document'), (req, res) => {

  const hash = generateHash(req.file.path);

  db.run(`INSERT INTO Documents (file_name, file_path, document_hash) VALUES (?, ?, ?)`,

    [req.file.originalname, req.file.path, hash]);

  res.send("Document uploaded successfully!");

});

// Verify API

app.post('/verify', upload.single('document'), (req, res) => {

  const newHash = generateHash(req.file.path);

```

```

db.get(`SELECT document_hash FROM Documents WHERE file_name = ?`,

[req.file.originalname],

(err, row) => {

    if (row && row.document_hash === newHash) {

        res.send("Document is ORIGINAL");

    } else {

        res.send("Document is TAMPERED");

    }

});

});

app.listen(3000, () => console.log("Server running on port 3000"));

```

APPENDIX B: DATABASE DESIGN

This section describes the database structure used in the project. The database is implemented using SQLite.

```
CREATE TABLE Users (
```

```

    user_id INTEGER PRIMARY KEY AUTOINCREMENT,

    username TEXT,

    email TEXT,

    password TEXT

);

CREATE TABLE Documents (

    document_id INTEGER PRIMARY KEY AUTOINCREMENT,

    file_name TEXT,

    file_path TEXT,

    document_hash TEXT,

    upload_date DATETIME DEFAULT CURRENT_TIMESTAMP

);

```

```

CREATE TABLE Blockchain_Records (

    block_id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```
document_id INTEGER,  
  
hash_value TEXT,  
  
previous_hash TEXT,  
  
timestamp DATETIME DEFAULT CURRENT_TIMESTAMP  
  
);
```

```
CREATE TABLE Verification_Records (  
  
verification_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
document_id INTEGER,  
  
uploaded_hash TEXT,  
  
stored_hash TEXT,  
  
status TEXT,  
  
verification_date DATETIME DEFAULT CURRENT_TIMESTAMP  
  
);
```

B.2 Explanation

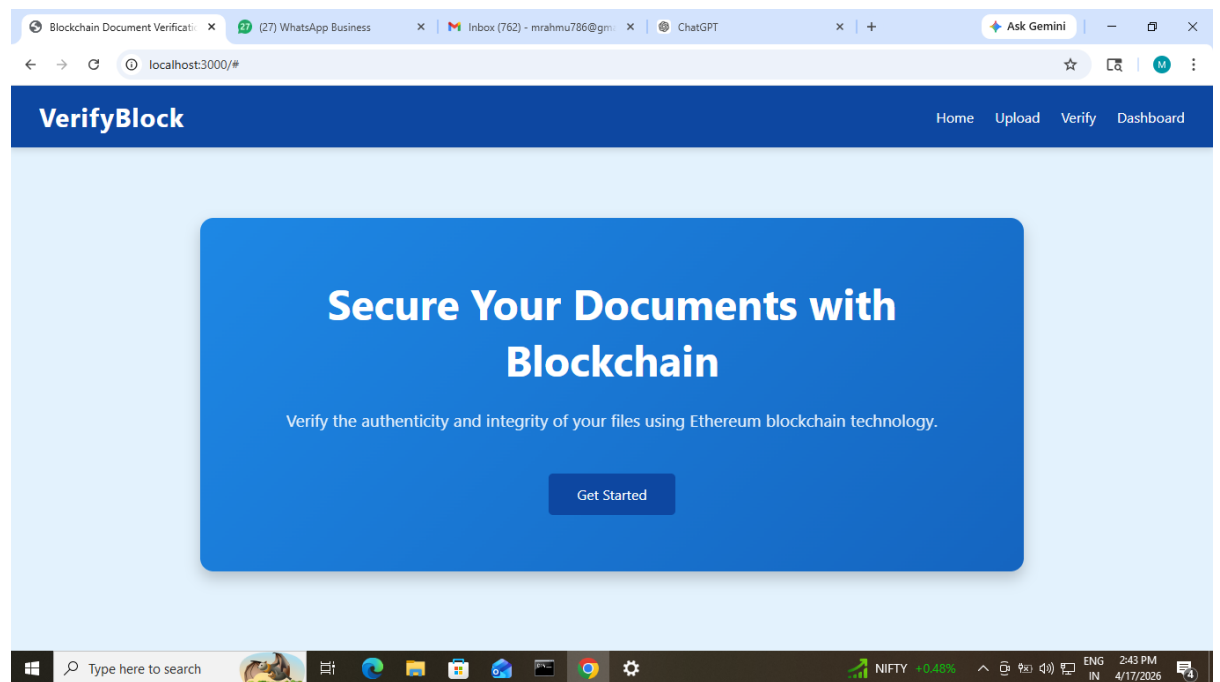
- **Users Table** → Stores user details
- **Documents Table** → Stores uploaded documents

- **Blockchain Table** → Stores hash values
- **Verification Table** → Stores verification results

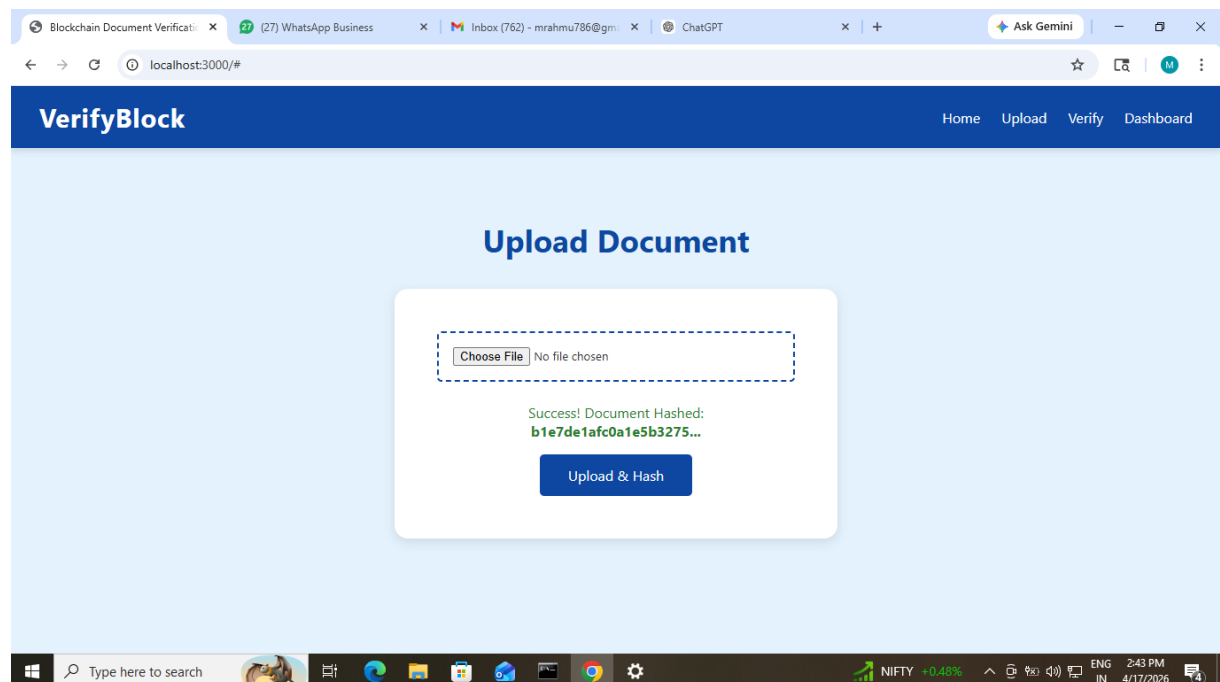
APPENDIX C: SYSTEM SCREENSHOTS

This section contains screenshots of the working application.

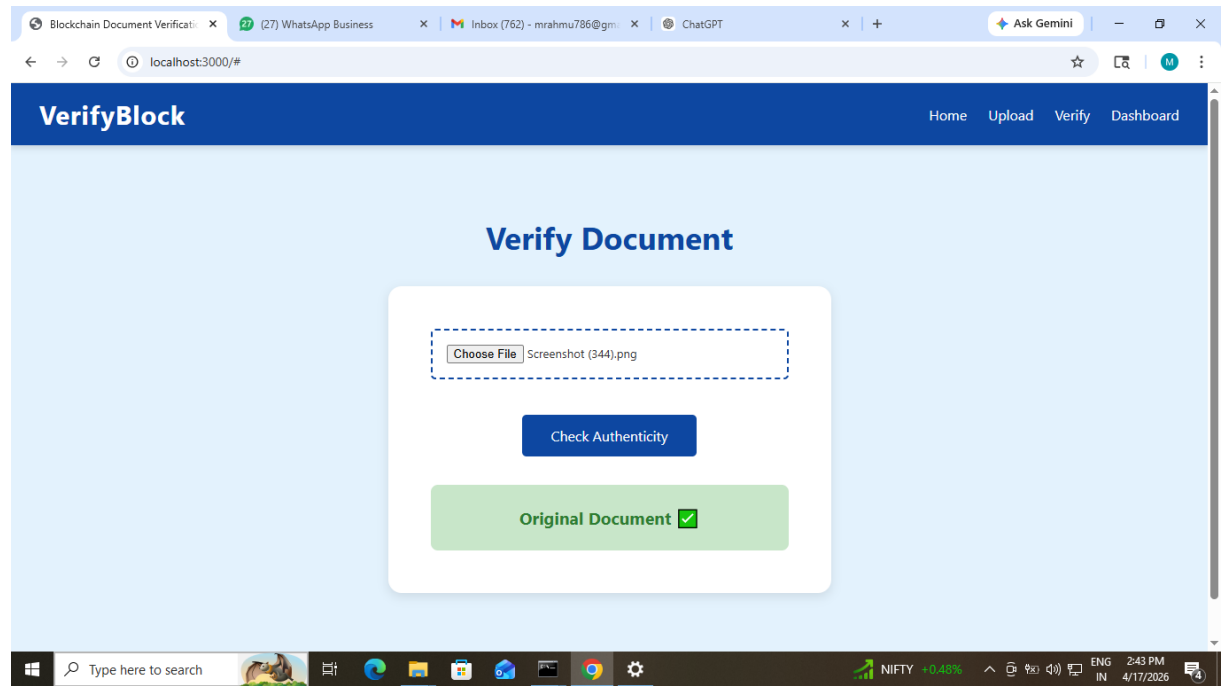
C.1 Home Page



C.2 Document Upload Page



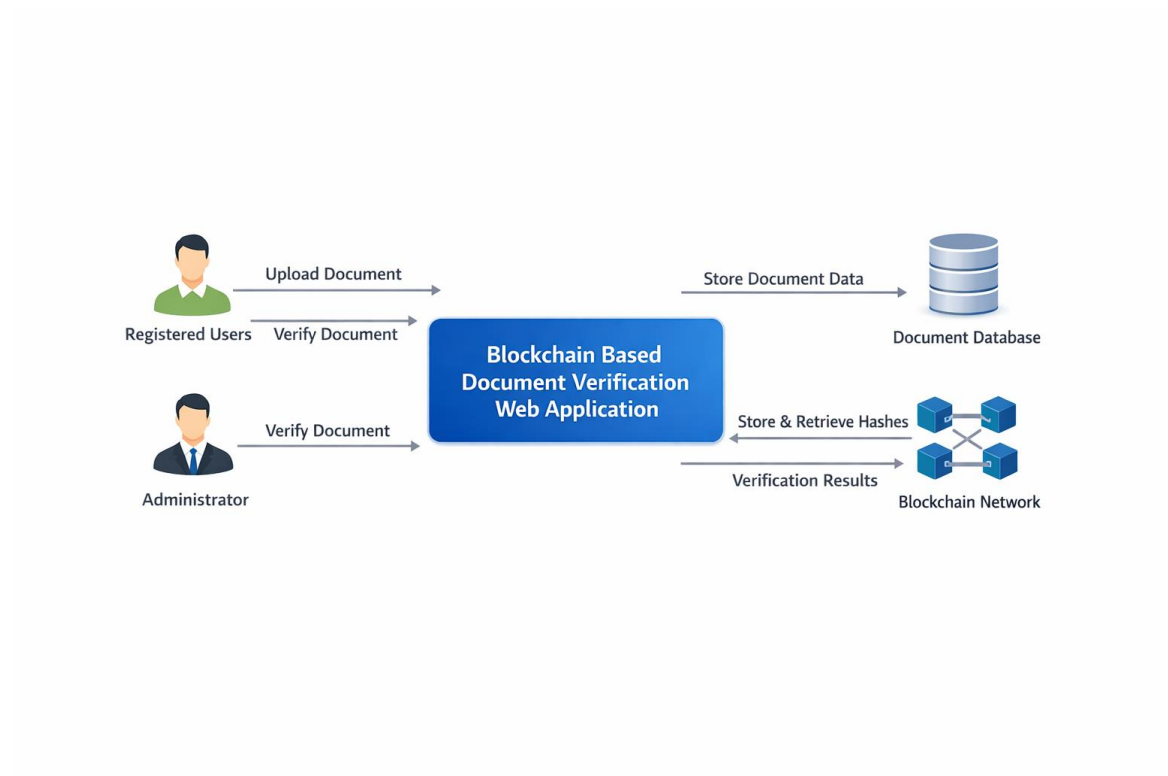
C.3 Verification Result Page



APPENDIX D: SYSTEM DIAGRAMS

This section includes all diagrams used in the project.

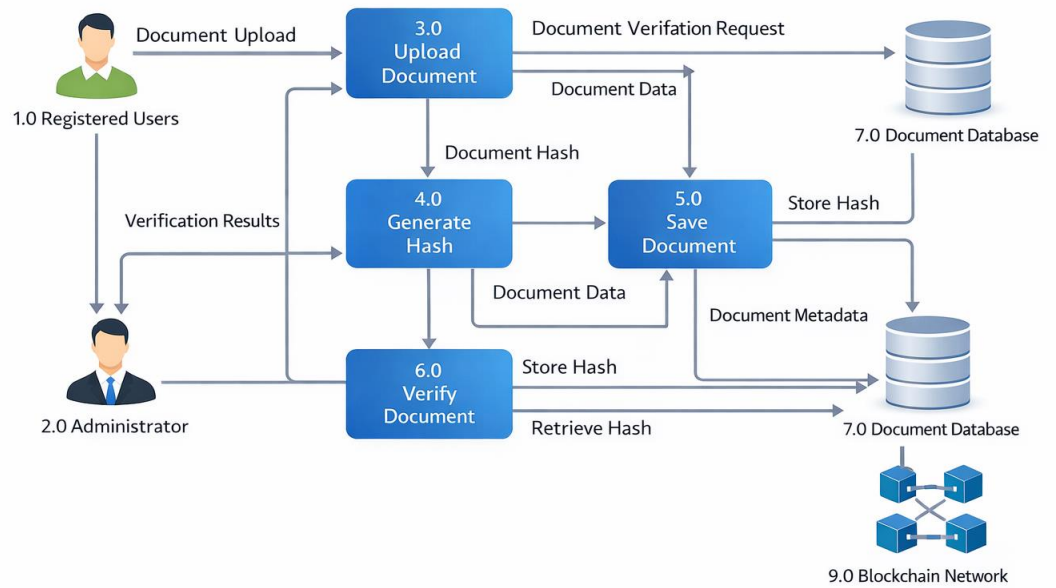
D.1 Context Diagram



D.2

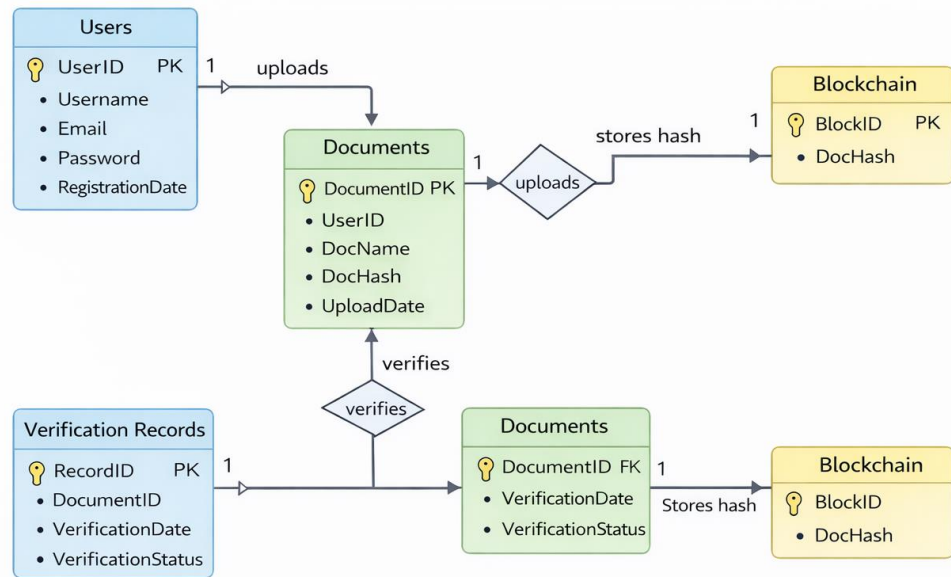
Data Flow Diagram (DFD)

Data Flow Diagram – Blockchain Based Document Verification Web Application

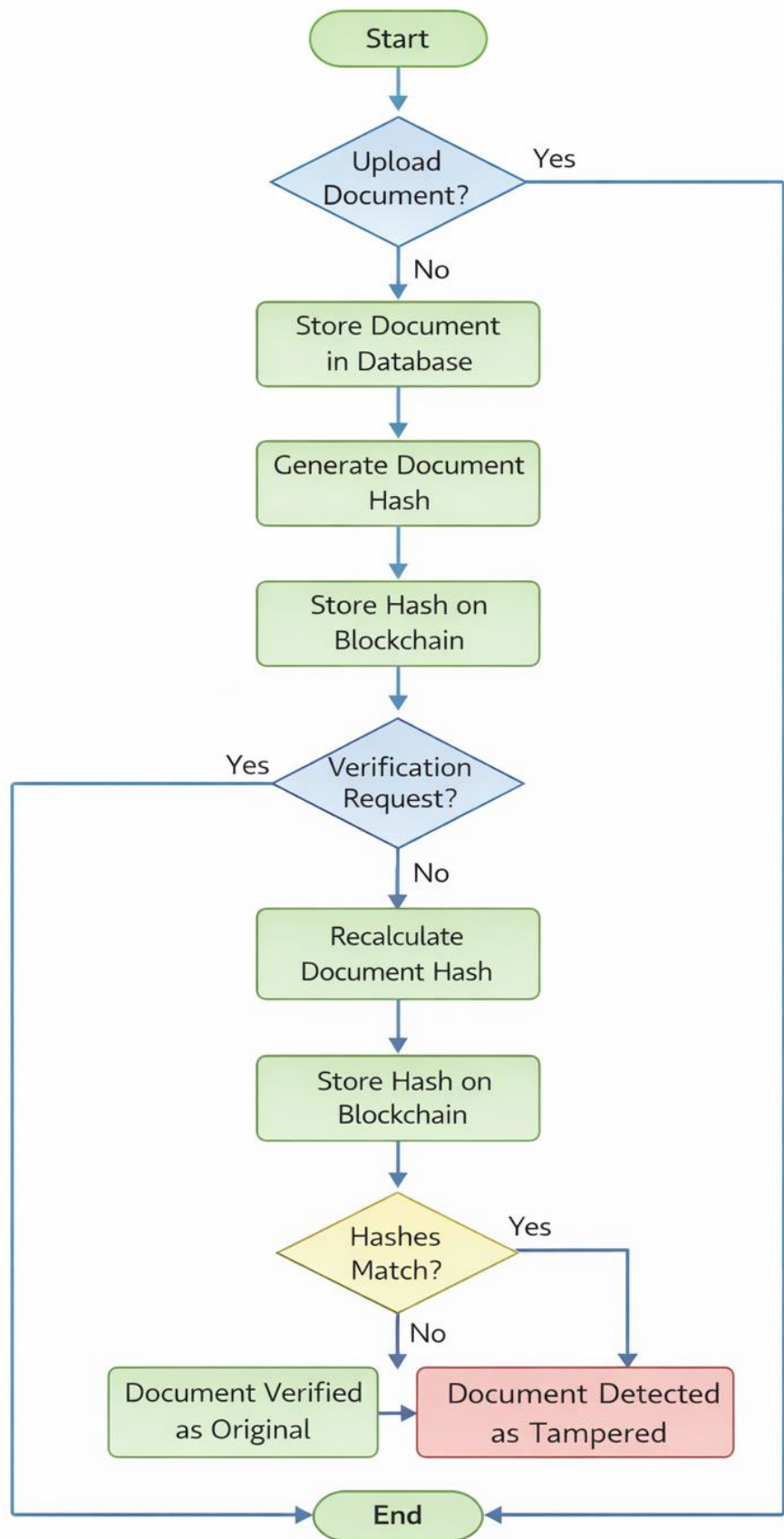


D.3 Entity Relationship Diagram (ERD)

Entity Relationship Diagram –
Blockchain Based Document Verification Web Application



D.4 Flowchart



APPENDIX E: SAMPLE DATA AND OUTPUT

E.1 Sample Hash

Original	Hash:
----------	-------

a3f5c9d2b8e4f7a6c1d9e0...

Modified	Hash:
----------	-------

b7a9d3e4f6c2a1e8d0f5...

E.2 Sample Output

Case	1:
------	----

Document	is	ORIGINAL
----------	----	----------

Case	2:
------	----

Document is TAMPERED

TEAM DETAILS

Team Member 1

Name: Mohd Ahmad Baqai (Team Leader)

Enrollment Number: 2300102880

Mail: mabaqai@student.iul.ac.in

Contact: 6392045974

Role In Project : Team Leader and Blockchain Developer

Responsibilities:

- Responsible for overall project management, coordination, and communication.
- Responsible for designing, developing, and testing the blockchain component and smart contracts.

Team Member 2

Name: Mohd. Farhan

Enrollment Number: 2300103452

Mail: mabaqai@student.iul.ac.in

Contact: 6392045974

Role In Project : Web Application Developer and Database Administrator

Responsibilities:

- Responsible for designing, developing and testing the web application's front-end and back-end.
- Responsible for designing, implementing and managing the project database.

Team Member 2

Name: Mohd. Taha

Enrollment Number: 2300100537

Mail: mohdtthaa@student.iul.ac.in

Contact: 7991227096

Role In Project : Testing and Quality Assurance

Responsibilities:

- Responsible for developing test plans, conducting testing, and ensuring the quality of the system..

.